

LINUX SYSTEM PROGRAMMING · 综合编程练习

Linux 系统编程

综合编程题 10 道（第二辑）

— 信号（Signal）专题 —

覆盖知识点：信号概述与四要素 · kill/raise/abort/alarm/setitimer
signal 与 sigaction 信号注册 · 信号集操作
sigprocmask 信号阻塞 · sigpending 未决信号集

题目来源：有道云笔记《DAY05-信号》知识点整理与设计
与第一辑（shell/系统调用/进程/exec）不重复

2026 年 8 月

题目说明与知识点覆盖表

1. 共 10 道编程题，聚焦信号（Signal）专题，由易到难递进，第 10 题为综合应用。
2. 与第一辑（shell/系统调用/进程/exec）内容不重复，本辑重点考察信号机制与信号处理函数注册。
3. 其中第 5、6 题为信号处理函数注册的核心题目（signal 与 sigaction 对比），建议重点练习。
4. 建议用时 90 分钟，每题 10 分，总分 100 分。

题号	题目	核心知识点
1	kill 信号发送与子进程控制	kill · fork · SIGINT · wait
2	raise 自发信号与信号处理	raise · signal · 自杀信号
3	alarm 定时器与超时退出	alarm · SIGALRM · signal注册
4	setitimer 循环定时任务	setitimer · sigaction · 周期定时
5	signal 注册 Ctrl+C 优雅退出	signal · SIGINT · malloc/free
6	sigaction 注册多个信号	sigaction · sa_mask · sa_flags
7	信号集操作综合练习	sigset_t · add/del/ismember
8	sigprocmask 信号阻塞与解除	SIG_BLOCK · SIG_UNBLOCK
9	sigpending 读取未决信号集	sigpending · 位图 · 信号状态
10	综合：父子进程信号协作	fork+kill+sigaction+wait

题目 1 kill 信号发送与子进程控制

考察知识点 | kill · fork · SIGINT · wait · 信号发送

【题目要求】

编写程序，父进程 fork 创建子进程后，向子进程发送 SIGINT 信号控制其退出：

1. 子进程循环打印"正在运行..."，每隔 1 秒打印一次，共 10 次；
2. 父进程等待 3 秒后，使用 kill 向子进程发送 SIGINT（2 号信号）；
3. 父进程使用 wait 回收子进程，并通过 WIFEXITED / WIFSIGNALED 判断子进程是正常退出还是被信号杀死；
4. 分别打印子进程的退出方式和退出状态/终止信号编号。

【参考代码框架】

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <signal.h>

int main() {
    pid_t pid = fork();
    if (pid == 0) { /* 子进程 */
        for (int i = 1; i <= 10; i++) {
            printf("正在运行... (%d)\n", i);
            sleep(1);
        }
        exit(0);
    } else { /* 父进程 */
        sleep(3);
        kill(pid, SIGINT); /* 向子进程发送 SIGINT */
        int status;
        wait(&status);
        if (WIFEXITED(status))
            printf("子进程正常退出，状态=%d\n", WEXITSTATUS(status));
        else if (WIFSIGNALED(status))
            printf("子进程被信号%d杀死\n", WTERMSIG(status));
    }
    return 0;
}
```

【提示】

提示：kill(pid, sig) 向指定 PID 的进程发送信号；WIFSIGNALED(status) 为真表示被信号终止，WTERMSIG(status) 取出终止信号编号；WIFEXITED(status) 为真表示正常退出。

题目 2 raise 自发信号与信号处理

考察知识点 | raise · signal · 自杀信号 · 信号注册

【题目要求】

编写程序，使用 raise 向自己发送信号，并用 signal 注册自定义处理函数：

1. 使用 signal 注册 SIGUSR1（10 号信号）的自定义处理函数，函数内打印"收到 SIGUSR1"；
2. 主进程打印自己的 PID 后，使用 raise(SIGUSR1) 向自己发送信号；
3. 观察：raise 之后是先执行信号处理函数还是先继续执行主程序？为什么？
4. 再用 raise(SIGTERM) 发送 15 号信号（未注册自定义处理），观察进程行为。

【参考代码框架】

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig) {
    printf("收到信号 %d (SIGUSR1)\n", sig);
}

int main() {
    signal(SIGUSR1, handler); /* 注册自定义处理 */
    printf("我的 PID 是 %d\n", getpid());

    raise(SIGUSR1);          /* 向自己发 SIGUSR1 */
    printf("raise 之后继续执行\n");

    raise(SIGTERM);          /* 向自己发 SIGTERM，默认终止 */
    printf("这行不会执行\n");
    return 0;
}
```

【提示】

提示：raise(sig) 等价于 kill(getpid(), sig)，向自己发送信号；信号处理函数执行完毕后，主程序从被中断的位置继续执行。SIGTERM 默认动作是终止进程。

题目 3 alarm 定时器与超时退出

考察知识点 | alarm · SIGALRM · 定时终止 · signal注册

【题目要求】

使用 alarm 函数实现一个超时控制程序：

1. 使用 signal 注册 SIGALRM (14 号信号) 的处理函数，函数内打印"时间到！ "；
2. 调用 alarm(5) 设置 5 秒定时器；
3. 主程序进入循环打印倒计时（5, 4, 3, 2, 1），每秒打印一次；
4. 5 秒后 SIGALRM 触发，执行信号处理函数后进程退出；
5. 思考：如果在第 3 秒时调用 alarm(0)，会发生什么？

【参考代码框架】

```
#include <stdio.h>
#include <unistd.h>
#include <signal.h>

void timeout_handler(int sig) {
    printf("\n时间到！ 信号 %d 触发\n", sig);
    _exit(0);
}

int main() {
    signal(SIGALRM, timeout_handler);
    alarm(5);          /* 5 秒后发送 SIGALRM */

    for (int i = 5; i > 0; i--) {
        printf("倒计时: %d\n", i);
        sleep(1);
    }
    /* 如果 alarm 没被取消，5 秒后 handler 会自动执行并退出 */
    return 0;
}
```

【提示】

提示：alarm(seconds) 在 seconds 秒后向进程发送 SIGALRM；alarm(0) 表示取消之前设置的闹钟，返回值是上一个闹钟的剩余秒数。

题目 4 setitimer 循环定时任务

考察知识点 | setitimer · ITIMER_REAL · sigaction · 周期定时 · struct itimerval

【题目要求】

使用 setitimer 实现周期性定时任务，每 2 秒触发一次：

1. 定义信号处理函数，每次触发打印当前触发次数和系统时间；
2. 使用 sigaction（而非 signal）注册 SIGALRM 的处理函数；
3. 配置 struct itimerval：it_value 设为 3 秒（首次触发延迟），it_interval 设为 2 秒（后续周期）；
4. 调用 setitimer(ITIMER_REAL, ...) 启动定时器；
5. 主程序进入 while(1) 循环，等待信号触发，打印 5 次后用 Ctrl+C 退出。

【参考代码框架】

```
#include <stdio.h>
#include <signal.h>
#include <sys/time.h>
#include <time.h>
#include <unistd.h>

int count = 0;

void timer_handler(int sig) {
    count++;
    time_t t = time(NULL);
    printf("第 %d 次触发, 时间: %s", count, ctime(&t));
}

int main() {
    struct sigaction act;
    act.sa_handler = timer_handler;
    sigemptyset(&act.sa_mask);
    act.sa_flags = 0;
    sigaction(SIGALRM, &act, NULL);

    struct itimerval timer;
    timer.it_value.tv_sec = 3;    /* 首次 3 秒后触发 */
    timer.it_value.tv_usec = 0;
    timer.it_interval.tv_sec = 2; /* 后续每 2 秒触发 */
    timer.it_interval.tv_usec = 0;

    setitimer(ITIMER_REAL, &timer, NULL);
    printf("定时器已启动, 首次 3 秒后触发, 之后每 2 秒一次\n");

    while (1) {
        sleep(1);
    }
    return 0;
}
```

【提示】

提示：ITIMER_REAL 计算自然时间，到时发送 SIGALRM；it_value 是首次触发时间，it_interval 是后续重复周期；struct timeval 包含 tv_sec（秒）和 tv_usec（微秒）。

题目 5 signal 注册 Ctrl+C 优雅退出与资源释放

考察知识点 | signal · SIGINT · malloc/free · 优雅退出 · 资源清理

【题目要求】

编写程序，使用 signal 注册 SIGINT 的处理函数，实现 Ctrl+C 时的优雅退出：

1. 在主程序中 malloc 分配一块内存，打印其地址；
2. 使用 signal(SIGINT, my_free) 注册处理函数，函数内检查指针是否为 NULL，非空则 free 并置 NULL，打印"资源已释放，再见！"；
3. 主程序进入 while(1) 循环，每 2 秒打印一次内存地址；
4. 运行程序后按 Ctrl+C，观察处理函数是否被调用、内存是否被释放；
5. 再次按 Ctrl+C，观察"资源已释放"的提示（因为指针已被置 NULL）。

【参考代码框架】

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

int *addr = NULL;

void my_free(int sig) {
    if (addr != NULL) {
        free(addr);
        addr = NULL;
        printf("资源已释放，再见！\n");
    } else {
        printf("资源已经释放过了\n");
    }
}

int main() {
    addr = (int *)malloc(sizeof(int));
    signal(SIGINT, my_free);

    while (1) {
        printf("堆区内存地址: %p\n", addr);
        sleep(2);
    }
    free(addr);
    return 0;
}
```

【提示】

提示：signal(SIGINT, handler) 注册后，按下 Ctrl+C 不再默认终止进程，而是调用 handler；在 handler 中做 free + 置 NULL 是典型的资源清理模式。注意：signal 在不同 Unix 版本行为不一致，实际工程推荐用 sigaction。

题目 6 sigaction 注册多个信号处理函数

考察知识点 | sigaction · struct sigaction · sa_handler · sa_mask · sa_flags · 多信号注册

【题目要求】

使用 sigaction 同时注册两个信号的自定义处理函数，对比 signal 的优势：

1. 注册 SIGINT (Ctrl+C) 处理函数：打印"收到 SIGINT"并记录触发次数，触发 3 次后恢复默认动作终止进程；
2. 注册 SIGQUIT (Ctrl+\) 处理函数：打印"收到 SIGQUIT"并退出进程；
3. 在 SIGINT 的处理期间，使用 sa_mask 屏蔽 SIGQUIT，确保处理函数不被打断；
4. 主程序进入 while(1) 循环等待信号；
5. 运行程序，分别按 Ctrl+C 和 Ctrl+\ 测试，观察行为差异。

【参考代码框架】

```

#include <stdio.h>
#include <signal.h>
#include <unistd.h>

int count = 0;

void int_handler(int sig) {
    count++;
    printf("收到 SIGINT (第 %d 次)\n", count);
    if (count >= 3) {
        printf("已达 3 次, 恢复默认动作\n");
        signal(SIGINT, SIG_DFL); /* 恢复默认 */
    }
}

void quit_handler(int sig) {
    printf("收到 SIGQUIT, 退出进程\n");
    _exit(0);
}

int main() {
    struct sigaction act_int;
    act_int.sa_handler = int_handler;
    sigemptyset(&act_int.sa_mask);
    sigaddset(&act_int.sa_mask, SIGQUIT); /* 处理 SIGINT 时屏蔽 SIGQUIT */
    act_int.sa_flags = 0;
    sigaction(SIGINT, &act_int, NULL);

    struct sigaction act_quit;
    act_quit.sa_handler = quit_handler;
    sigemptyset(&act_quit.sa_mask);
    act_quit.sa_flags = 0;
    sigaction(SIGQUIT, &act_quit, NULL);

    printf("PID=%d, 按 Ctrl+C 测试 SIGINT, Ctrl+\ 测试 SIGQUIT\n", getpid());
    while (1) {
        sleep(1);
    }
    return 0;
}

```

【提示】

提示：sigaction 结构体的 sa_mask 字段指定在信号处理函数执行期间需要额外屏蔽的信号集；sa_flags 设为 0 使用默认属性；SA_RESETHAND 可使信号处理完后恢复默认动作。

题目 7 信号集操作综合练习

考察知识点 | sigset_t · sigemptyset · sigfillset · sigaddset · sigdelset · sigismember

【题目要求】

编写程序，练习信号集（sigset_t）的基本操作：

1. 创建两个信号集 set1 和 set2；
2. set1 使用 sigemptyset 清空，然后添加 SIGINT(2)、SIGTERM(15)、SIGUSR1(10) 三个信号；
3. set2 使用 sigfillset 填充所有信号，然后删除 SIGKILL(9) 和 SIGSTOP(19)（这两个不可被屏蔽）；
4. 遍历 1~31 号信号，分别判断它们是否在 set1 和 set2 中，打印结果（在→1，不在→0）；
5. 从 set1 中删除 SIGTERM，再次判断 SIGTERM 是否在 set1 中。

【参考代码框架】

```
#include <stdio.h>
#include <signal.h>

void print_set(sigset_t *set, const char *name) {
    printf("%s: ", name);
    for (int i = 1; i <= 31; i++) {
        printf("%d", sigismember(set, i));
    }
    printf("\n");
}

int main() {
    sigset_t set1, set2;

    /* set1: 空集 + 添加3个信号 */
    sigemptyset(&set1);
    sigaddset(&set1, SIGINT);
    sigaddset(&set1, SIGTERM);
    sigaddset(&set1, SIGUSR1);
    print_set(&set1, "set1");

    /* set2: 全集 - 删除2个不可屏蔽信号 */
    sigfillset(&set2);
    sigdelset(&set2, SIGKILL);
    sigdelset(&set2, SIGSTOP);
    print_set(&set2, "set2");

    /* 从 set1 删除 SIGTERM */
    sigdelset(&set1, SIGTERM);
    printf("删除 SIGTERM 后, set1 中 SIGTERM 存在: %d\n",
        sigismember(&set1, SIGTERM));

    return 0;
}
```

【提示】

提示: sigset_t 是信号集类型; sigismember 返回 1 表示信号在集合中, 0 表示不在; SIGKILL(9) 和 SIGSTOP(19) 不能被阻塞、不能被自定义捕获。

题目 8 sigprocmask 信号阻塞与解除

考察知识点 | sigprocmask · SIG_BLOCK · SIG_UNBLOCK · 信号屏蔽 · 延迟处理

【题目要求】

编写程序，使用 sigprocmask 阻塞 SIGINT 信号 5 秒后解除：

1. 创建信号集，添加 SIGINT，使用 sigprocmask(SIG_BLOCK, ...) 将其加入阻塞集；
2. 打印"SIGINT 已阻塞，请在 5 秒内按 Ctrl+C 测试"；
3. sleep(5) 期间用户按 Ctrl+C，观察：进程不会终止（信号被延迟）；
4. 5 秒后使用 sigprocmask(SIG_UNBLOCK, ...) 解除阻塞，观察：之前阻塞的 SIGINT 被立即处理；
5. 使用 signal 或 sigaction 注册 SIGINT 处理函数，让解除阻塞后打印"收到被延迟的 SIGINT"。

【参考代码框架】

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig) {
    printf("\n收到信号 %d（可能是之前被阻塞的）\n", sig);
}

int main() {
    signal(SIGINT, handler);

    sigset_t set;
    sigemptyset(&set);
    sigaddset(&set, SIGINT);

    /* 阻塞 SIGINT */
    sigprocmask(SIG_BLOCK, &set, NULL);
    printf("SIGINT 已阻塞，5 秒内按 Ctrl+C 不会终止\n");

    sleep(5);

    /* 解除阻塞 */
    printf("解除阻塞...\n");
    sigprocmask(SIG_UNBLOCK, &set, NULL);
    printf("阻塞已解除，如果有被延迟的信号将被处理\n");

    while (1) {
        sleep(1);
    }
    return 0;
}
```

【提示】

提示：sigprocmask 修改进程的信号屏蔽字（阻塞信号集）；SIG_BLOCK 添加信号到阻塞集，SIG_UNBLOCK 从阻塞集移除；被阻塞的信号不会丢失，解除阻塞后会被递送处理。普通信号（1~31）多次发送只保留一次（不排队）。

题目 9 sigpending 读取未决信号集

考察知识点 | sigpending · 未决信号集 · 位图 · sigprocmask · 信号状态观察

【题目要求】

编写程序，阻塞 SIGINT 和 SIGTSTP 后，读取并显示未决信号集：

1. 使用 signal 或 sigaction 为 SIGINT 和 SIGTSTP 注册处理函数；
2. 使用 sigprocmask 阻塞这两个信号；
3. 进入 while(1) 循环，每 2 秒调用 sigpending 读取未决信号集；
4. 遍历 1~31 号信号，用 0/1 打印每个信号在未决集中是否存在（形成 31 位位图）；
5. 运行程序后，在另一个终端或按 Ctrl+C / Ctrl+Z 发送信号，观察未决位图中对应位从 0 变 1。

【参考代码框架】

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig) {
    printf("处理信号 %d\n", sig);
}

void show_pending(sigset_t *pending) {
    printf("未决信号集(1~31): ");
    for (int i = 1; i <= 31; i++) {
        printf("%d", sigismember(pending, i));
    }
    printf("\n");
}

int main() {
    signal(SIGINT, handler);
    signal(SIGTSTP, handler);

    sigset_t set;
    sigemptyset(&set);
    sigaddset(&set, SIGINT);
    sigaddset(&set, SIGTSTP);
    sigprocmask(SIG_BLOCK, &set, NULL);

    printf("SIGINT 和 SIGTSTP 已阻塞\n");
    printf("请按 Ctrl+C 或 Ctrl+Z 测试，观察未决位图变化\n");

    while (1) {
        sigset_t pending;
        sigemptyset(&pending);
        sigpending(&pending);
        show_pending(&pending);
        sleep(2);
    }
    return 0;
}
```

【提示】

提示：未决信号集是“已产生但尚未处理”的信号集合；`sigpending(set)` 将当前进程的未决信号集复制到 `set` 中；信号被处理后对应位自动清零。被阻塞的信号产生后会进入未决状态。

题目 10 综合：带信号处理的父子进程协作

考察知识点 | 综合 · fork · kill · signal · sigaction · wait · 父子通信

【题目要求】

综合运用 fork、kill、signal/sigaction、wait 实现父子进程间的信号协作：

1. 父进程 fork 创建子进程；
2. 子进程使用 sigaction 注册 SIGUSR1 的处理函数（收到后打印"父进程通知我"并计数）；
3. 子进程进入 while(1) 循环，每秒打印"子进程等待中..."；
4. 父进程每 2 秒使用 kill 向子进程发送 SIGUSR1，共发送 3 次；
5. 第 3 次发送后，父进程使用 kill 发送 SIGTERM 终止子进程，wait 回收并打印退出信息；
6. 父进程在发送信号期间也注册 SIGINT 处理函数，如果用户按 Ctrl+C 则提前终止子进程并退出。

【参考代码框架】


```

#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

pid_t child_pid;

void child_handler(int sig) {
    static int count = 0;
    count++;
    printf("子进程: 收到父进程通知 (第 %d 次)\n", count);
}

void parent_handler(int sig) {
    printf("\n父进程: 收到 Ctrl+C, 终止子进程\n");
    kill(child_pid, SIGTERM);
}

int main() {
    child_pid = fork();
    if (child_pid == 0) { /* 子进程 */
        struct sigaction act;
        act.sa_handler = child_handler;
        sigemptyset(&act.sa_mask);
        act.sa_flags = 0;
        sigaction(SIGUSR1, &act, NULL);

        while (1) {
            printf("子进程等待中...\n");
            sleep(1);
        }
    } else { /* 父进程 */
        signal(SIGINT, parent_handler);

        for (int i = 0; i < 3; i++) {
            sleep(2);
            kill(child_pid, SIGUSR1);
            printf("父进程: 已发送第 %d 次通知\n", i + 1);
        }

        sleep(1);
        kill(child_pid, SIGTERM);
        int status;
        wait(&status);
        if (WIFSIGNALED(status))
            printf("子进程被信号 %d 终止\n", WTERMSIG(status));
        else
            printf("子进程正常退出\n");
    }
    return 0;
}

```

【提示】

提示：SIGUSR1(10) 和 SIGUSR2(12)

是用户自定义信号，默认动作是终止进程，所以必须注册处理函数才能捕获；父子进程通过信号实现简单的异步通知，这是进程间通信（IPC）的一种方式。

评分参考

程序可正确编译并运行（gcc 无警告，无段错误）	4 分
功能完整，覆盖题目要求的全部功能点	3 分
信号处理函数注册正确，逻辑清晰（回调函数签名、参数使用）	2 分
代码规范、有注释、能回答思考题	1 分

注：第 5、6 题为信号注册核心题，建议重点考察 signal 与 sigaction 的区别（可移植性、sa_mask 屏蔽、sa_flags 行为控制）。

完成顺序建议：1→2→3→4→5→6→7→8→9→10，前 4 题是信号基础 API，5~6 题是信号注册核心，7~9 题是信号集与阻塞，第 10 题综合串联。